

SCOPE OF WORK

Executive Insight Agent

An AI agent that answers project, code, and team-health questions on demand — in plain language, with the “why”.

In one line. Replace the half-hour of manual dashboard-reading executives do today with a single question in a mobile app, answered by an agent that reasons over the same KPIs leadership already trusts.

Document control	
Prepared for	Executive Leadership (CEO and senior leaders)
Prepared by	[Project team name]
Version	1.0
Date	26 July 2026
Status	Draft for sign-off

1 Problem statement

Today, when a company leader wants to know how a project is really doing — is the code healthy, is delivery on track, and if something is slipping, why — there is no single place to ask. The information lives in four separate systems: GitHub, Jira and SonarQube on the engineering side, and Ngage for attendance and performance. It is surfaced through several Grafana dashboards. To get an answer, an executive has to open each dashboard, read the charts, and piece the story together by hand. This takes around thirty minutes for every question, it has to be repeated from scratch each time, and because nobody is joining the delivery data to the team data, the most important question — why is this happening — is the easiest one to miss. The people who feel this most are the CEO and senior leaders, who need quick, trustworthy answers and instead spend their time doing manual detective work.

2 Objectives

When this is delivered, the following will be true that is not true today:

- **Ask in plain language, answer in under a minute.** A leader opens a mobile app, types a question such as “How is Project X doing?” or “Why did last sprint slip?”, and gets a clear answer — down from roughly thirty minutes to under one.
- **Answers explain the why, not just the what.** The response connects delivery and code-quality signals with attendance and performance, so leaders see likely causes, not only numbers.
- **Grounded in the KPIs leadership already trusts.** The agent uses the same metric definitions that power the existing Grafana dashboards, so there is no second, competing version of the truth.
- **A genuine reasoning agent.** It plans each question, calls the right data tools, recovers gracefully when a tool fails, and remembers the conversation so follow-up questions simply work.
- **Near-zero running cost, no loss of quality.** A free-tier language model and a once-a-day data refresh keep costs down and put no live load on the existing analytics database.
- **Self-service, any hour.** Leaders get answers without pulling an analyst or engineer away from their work.

3 Solution overview

This section explains the approach and the pipeline at a high level; the diagrams carry the detail. It is included because the plan depends on it — the rest of the Scope of Work stays deliberately non-technical.

3.1 The problem, drawn

Today the data is scattered and a person is the integration layer. Every question means opening four systems and several dashboards and reasoning across them by hand.

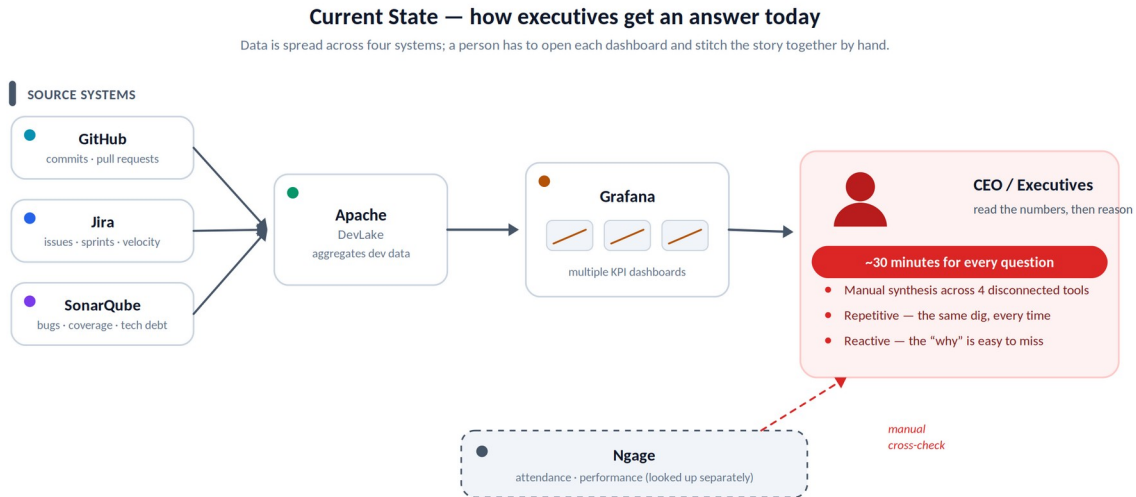


Figure 1. How executives get an answer today — scattered data, manual synthesis, the “why” easily missed.

3.2 The approach in plain terms

Two ideas do the heavy lifting. First, we do not make the agent run slow, live queries. Instead, once each morning we pre-compute the very same numbers the Grafana dashboards show and store them, already structured, in a small purpose-built database we call the Metrics Store. Second, we give a reasoning AI agent a set of simple tools to read that store — one tool per KPI area — so it can plan a question, fetch exactly what it needs, and explain the answer. The agent never touches the live analytics database, which keeps both cost and load down.

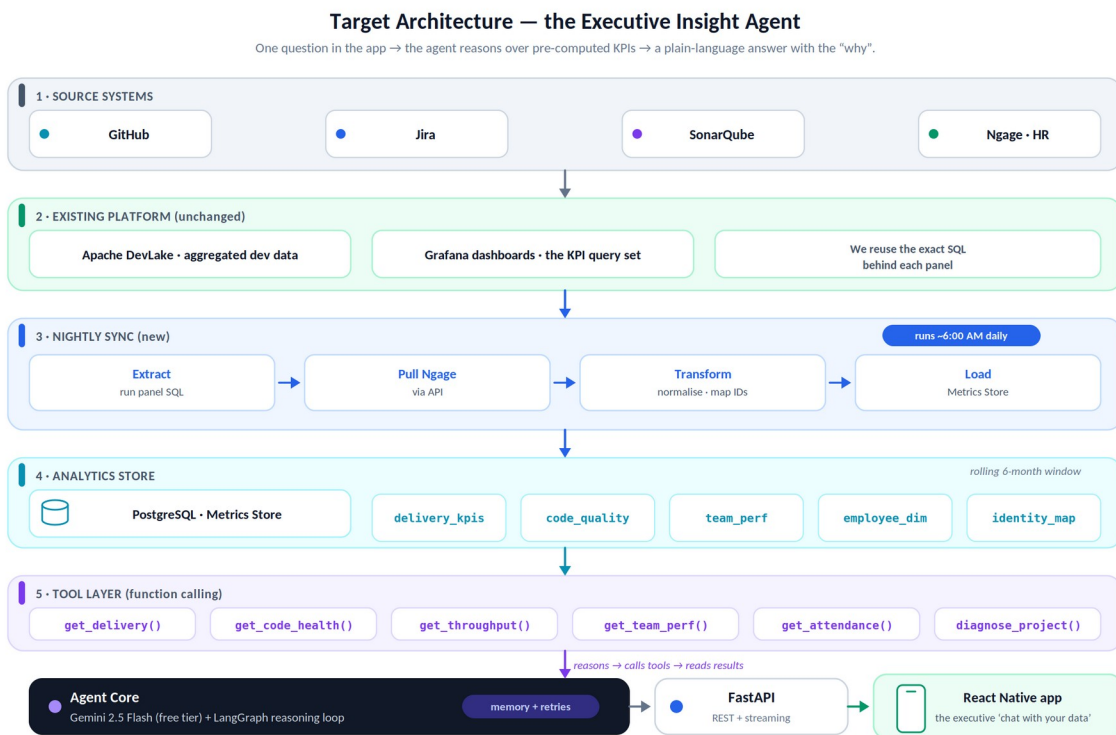


Figure 2. Target architecture — from the source systems, through the nightly sync and Metrics Store, to the agent and the executive’s app.

3.3 The nightly sync

A scheduled job runs before work hours. It reuses the exact SQL behind each Grafana panel (across all dashboards, driven by a small configurable registry so new dashboards can be added without code changes), pulls attendance and performance from Ngage, normalises everything, maps each person across Git, Jira and Ngage, and loads clean tables into the Metrics Store on a rolling six-month window. Freshness and row-count checks run at the end; if anything fails, the job raises an alert and keeps yesterday’s good snapshot rather than serving broken data.

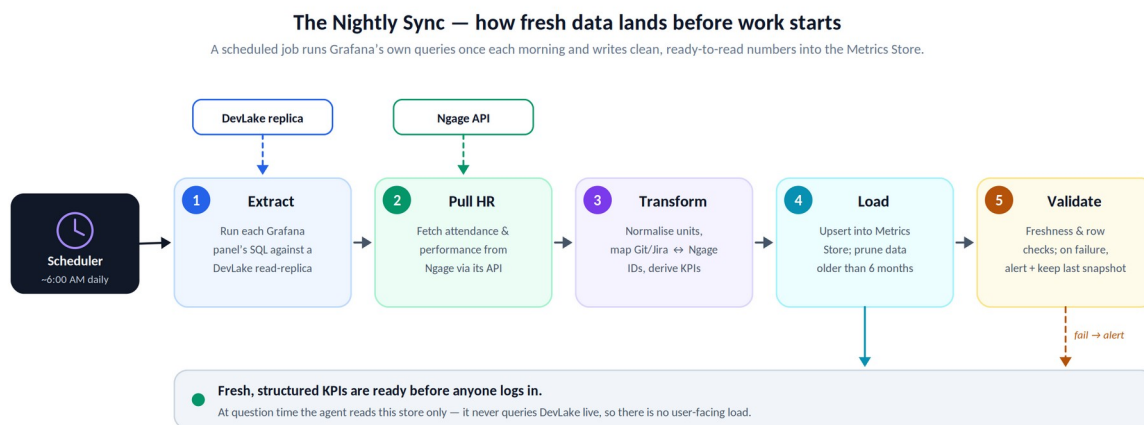


Figure 3. The nightly sync — fresh, structured KPIs are ready before anyone logs in, with no user-facing database load.

3.4 How the agent reasons

The agent works in a loop rather than a single shot. It breaks a question down and plans what data would answer it, picks and calls the right tool, and checks the result. If a tool call fails, it retries with back-off, then reformulates or falls back to partial data and clearly says what is missing — it does not crash or invent an answer. It keeps going until it has enough to respond, then composes a plain-language answer with the numbers that matter and the likely cause. The full conversation is held in memory, so follow-up questions keep their context. This is what makes it a proper agent, not a chatbot bolted onto a database.

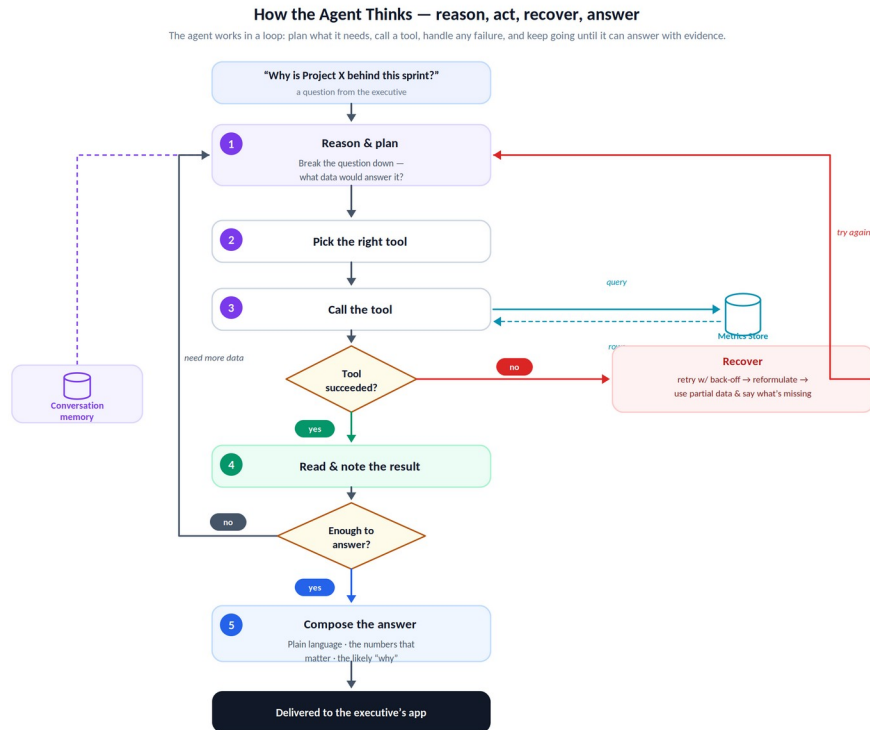


Figure 4. The reasoning loop — plan, act, recover from failure, and answer with evidence.

3.5 Technology choices

Each choice favours capability at the lowest sustainable cost, with a one-line reason.

Layer	Recommended choice	Why this fit
Reasoning model	Gemini 2.5 Flash (free tier), Flash-Lite as fallback	Capable, strong tool-calling and a 1M-token context on a genuinely free tier; the cheaper Flash-Lite handles simple steps.
Agent framework	LangGraph (Python)	Purpose-built for looping, stateful agents with tool-calling, retries and persistent conversation memory.
Backend	FastAPI (Python)	Fast, async, native response streaming, and a natural home for the agent alongside the existing Python models.
Metrics Store	PostgreSQL	Reliable, free and open-source; ideal for the structured KPI and employee tables.
Nightly sync	Scheduled job (cron / lightweight orchestrator)	Simple and predictable; one run a morning is all that is needed.
Mobile app	React Native (Expo)	One codebase, Android-first as requested, and quick to build a clean streaming chat UI; the same backend can serve a web client later.

On cost. A free-tier model plus a once-a-day batch keeps the running cost close to zero. Response caching, small structured tool outputs, and exponential back-off keep usage inside the free-tier rate limits. Where sensitive employee data is involved, we recommend a small paid allowance (see Risks); this is the one place where spending a little protects privacy.

4 Scope — version one

Version one delivers a working, end-to-end agent for a small group of leaders. Specifically:

- **Mobile chat app (Android).** A leader signs in and asks questions in natural language.
- **Streaming, plain-language answers.** Each answer includes the key numbers and the likely “why”, streamed as it is generated.
- **Core KPI coverage.** The metric areas already in Grafana: delivery and throughput (velocity, cycle time, pull-request throughput, deployment frequency), code health (bugs, coverage, tech debt and quality gate), and team signals (attendance, performance) from Ngage — subject to what those dashboards expose.
- **The flagship “diagnose” capability.** Ask “why is Project X behind?” and the agent checks delivery, code quality and team signals together and gives a single reasoned answer.
- **A real reasoning agent.** Step-by-step planning, tool-calling, failure recovery, and conversation memory for follow-ups.
- **The Metrics Store and nightly sync.** Reproduces Grafana’s KPIs across all configured dashboards on a rolling six-month window.
- **Identity mapping.** Links a person’s Git and Jira activity to their Ngage record.
- **Access control and light auditing.** Sign-in with a leadership-only role check, and a simple log of questions asked.
- **Admin configuration.** A small registry to choose which dashboards and panels the sync pulls, so new dashboards can be added without code changes.

5 Out of scope (for now)

Deciding what we are not building is as important as deciding what we are. The following are deliberately excluded from version one:

- Writing back to any system. The agent is strictly read-only — it will not create Jira tickets or change data anywhere.
- Forecasting or automated scoring of individuals. No predictive models and no automatic performance ratings of people.
- iOS build and a full web application. Android is the version-one target; the architecture leaves the door open for both later.
- Real-time metrics. Data refreshes once each morning; sub-daily or live streaming is not included.
- Replacing Grafana or DevLake. We complement the existing stack, we do not replace it.
- Training or fine-tuning a custom model. We use the model as-is.
- Company-wide rollout and enterprise identity. No non-leadership user tiers, single sign-on, or org-wide permissioning in v1.
- Alerts and notifications. “Tell me when X drops” is a natural version-two feature, not part of this scope.
- Long-term historical warehousing. Data beyond the rolling six-month window is not retained.

6 Stakeholders

Main user: the CEO and senior leaders. **Signs off:** the executive sponsor, with the product owner confirming scope is met.

Role	Who	Involvement
Executive sponsor / main user	CEO / senior leadership	Primary user of the app; approves the vision and signs off on acceptance.
Product / project owner	[Team lead]	Owns scope, priorities and acceptance criteria; day-to-day decisions.
Delivery team	[Engineering team]	Builds the pipeline, agent, backend and app.
DevLake / Grafana owner	[Data / platform admin]	Provides read access (ideally a replica) and the SQL behind each panel.
Ngage / HR owner	[HR lead]	Provides Ngage API access and approves which employee data may be used.
IT / Security	[IT / Security]	Approves data access, hosting, and the model data-handling decision.
UAT reviewers	2-3 named executives	Validate that answers are correct and useful during acceptance.

7 Deliverables

At the end of the engagement we will hand over:

- This Scope of Work, signed.
- The Metrics Store schema and the nightly sync job, with source code and instructions to deploy and run it.
- The KPI tool library — one tool per KPI area plus the “diagnose” tool.
- The reasoning agent service (Gemini + LangGraph) with tool-calling, failure recovery and conversation memory.
- The FastAPI backend — sign-in and role check, a streaming chat endpoint, conversation storage and the audit log.
- The React Native Android app, delivered as an installable build (APK).
- Configuration for dashboard/panel registration and identity mapping.
- Documentation — a setup and run guide, short architecture notes, and an operate-and-extend runbook.
- A recorded demonstration and the results of user-acceptance testing.

8 Timeline

A six-week plan, sequenced so a working end-to-end slice is demonstrable by Week 3 and the remaining weeks deepen reasoning, build the app, and harden for hand-over. Durations are adjustable to team size.

Week	Focus	Key deliverable
1	Foundations & data discovery	Get access via DevLake, audit every Grafana panel and export its SQL, review the Ngage API, design the Metrics Store schema, set up the environment. → Agreed KPI list, schema, working dev environment.
2	Data pipeline & Metrics Store	Build the nightly sync, load the first delivery and code-quality KPIs, integrate Ngage, implement identity mapping, six-month pruning and freshness checks. → Store populated with real KPIs; sync runs green.
3	Tool layer + agent skeleton	Build the KPI tools, wire up Gemini and LangGraph, connect the first end-to-end path. → Thin slice: the agent answers one real question end-to-end (live demo).
4	Reasoning depth & recovery	Build the “why is Project X behind?” diagnostic flow, add tool-failure handling and conversation memory, evaluate against a set of real questions. → Reliable multi-step diagnostic answers.
5	Backend API + React Native app	FastAPI streaming endpoint, auth and role check, conversation store and audit log; React Native chat UI with streaming, history and sign-in. → A working app talking to the agent.
6	Integration, UAT & hand-over	End-to-end hardening, rate-limit and performance tuning, security review, documentation, executive UAT, demo and sign-off. → Accepted v1 and all deliverables handed over.

Delivery Plan — six weeks to a working agent

A thin end-to-end slice is live by Week 3; the rest of the plan deepens reasoning, builds the app, and hardens for hand-over.

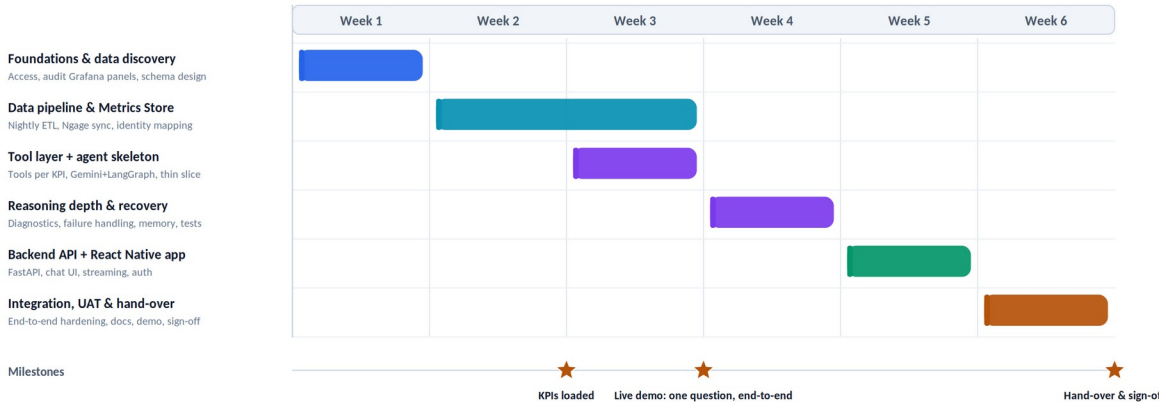


Figure 5. The six-week delivery plan, with milestones for KPIs loaded, the first live demo, and hand-over.

9 Assumptions and risks

9.1 Assumptions

The plan assumes the following are true. If any turns out not to be, the timeline or scope may need to change.

- DevLake already holds reliable, current data from GitHub, Jira and SonarQube, and we can get read access (ideally a read-replica) plus the SQL behind each Grafana panel.
- Grafana's existing KPI definitions are the agreed source of truth; we reproduce them rather than invent new metrics.
- Ngage exposes an API or export for attendance and performance, and HR and leadership approve using that data for these insights.
- A dependable key exists, or can be built, to match a person across Git, Jira and Ngage (for example, work email).
- Gemini's free tier remains available with tool-calling and limits sufficient for a small group of leaders; a small paid tier is acceptable if privacy or limits require it.
- Once-a-morning data is timely enough for leadership decisions in version one.
- A few named executives are available for acceptance testing and sign-off within the timeline, and basic hosting (a small server or cloud) is available.

9.2 Risks

Risk	Likelihood	Impact	Mitigation
Free-tier model may use prompts to improve the vendor's products — inappropriate for attendance and performance data.	Medium	High	Send the model minimal, aggregated data — never raw personal records; for anything sensitive, move to a paid tier or Vertex AI, which do not train on your data. Budget a small allowance.
Free-tier rate limits (a few requests per minute) under bursty use.	Medium	Medium	Caching, small payloads, exponential back-off and a simple queue; upgrade the tier if the pilot outgrows it.
Some Grafana queries are complex or DevLake access is restricted.	Medium	Medium	Audit panels in Week 1, start with the highest-value KPIs, and agree read-replica access up front.
Identity mapping is incomplete (different handles across tools).	Medium	Medium	Build a mapping table with a manual override; flag unmapped activity rather than guessing.
"Why" answers imply judgments about individuals and can be misread.	Medium	High	Frame outputs as signals, not verdicts; keep a human in the loop; restrict access to leadership; add clear caveats and align with HR on acceptable use.
The agent gives a confident but wrong answer.	Medium	High	Ground every claim in tool data, show the numbers behind each answer, evaluate against a fixed question set, and prefer "I don't have that" over guessing.
Scope creep (alerts, forecasting, more users).	High	Medium	Hold the version-one scope in this document; capture requests in a version-two backlog.

Risk	Likelihood	Impact	Mitigation
Upstream data is stale or has gaps.	Medium	Medium	Freshness checks in the sync, a visible “data as of” timestamp, and alert-plus-last-good-snapshot on a failed run.

10 What “done” looks like

Version one is complete and accepted when the following are demonstrably true:

Acceptance criterion	Measure / target
A leader can install the app, sign in, and get correct, grounded answers to an agreed set of real questions.	≈ 10 questions answered correctly in UAT.
The flagship diagnostic works.	“Why is Project X behind?” returns a reasoned answer combining delivery, code-quality and team signals, matching an analyst’s conclusion.
Answers are fast.	Median time-to-answer under ~1 minute (vs ~30 minutes today).
The Metrics Store matches Grafana.	Nightly sync runs unattended; numbers reconcile with Grafana within an agreed tolerance.
Failures are handled gracefully.	Tool failures never crash the agent; it recovers or states clearly what is missing.
Cost stays controlled.	Within free-tier for the pilot group, or an agreed small budget.
Everything is handed over and signed off.	All deliverables in Section 7 delivered; executive sponsor signs off.

11 Sign-off

By signing below, the parties agree that this document accurately describes what will be built in version one, and what will not.

Executive sponsor

Name · Signature · Date

Product / project owner

Name · Signature · Date